

內容自動化發布工作流程解說

Drive → Apps Script → GitHub → Firebase Hosting → Client
可複用到其他「遠端 JSON / 靜態資料」專案 · 密鑰皆以隱碼標示

適用對象： 想把「AI / 人工產出的 JSON（或其他靜態資料）」自動送到公開 CDN，供 App 或網站抓取的維護者與學習者。

參考實作： 本倉庫（公開圖鑑 JSON CDN） — [jimmy77733/wandanle-catalog](https://github.com/jimmy77733/wandanle-catalog)

文件版本： 2026-08-01

授權： 可自由複製、改寫到你自己的專案；請勿把真實密鑰貼進公開文件。

0. 這份文件在說什麼？

許多專案需要「遠端資料檔」：例如圖鑑、題庫、設定表。理想流程是：

1. 內容生產者（人或 AI）產出一份合法 JSON
2. 系統自動驗證並寫入公開倉庫
3. CI 部署到 Hosting / CDN
4. 客戶端 App 定期下載最新檔

本文件以本倉庫實際採用的管線為範例，說明**為什麼這樣設計**、**每一段如何銜接**，以及你要複用時該替換成什麼。

隱私聲明： 文中所有密鑰、檔案 ID、私人帳號皆以隱碼標示（如 `.....`）。公開 URL、公開 repo 名稱可保留為可驗證的實例。

1. 為什麼不能「讓 AI 直接 push GitHub」？

常見限制（以 Gemini Spark 等助理為例）：

能力	通常可否	影響
讀寫使用者的 Google Drive	可以	適合當「內容暫存站」
持有 / 儲存 API Token	不行	不能把 PAT 寫進 Skill
對外 HTTP POST (webhook)	不行 / 不可靠	不能直打 GitHub API
<code>git push</code> 到任意 repo	不行	不能當正式發布者

因此正式發布必須由你控制的帳號與密鑰完成：Google Apps Script (Script Properties) + GitHub Actions Secrets。

2. 整套架構（一眼看懂）



職責切分原則：

- AI：只負責產出正確內容，寫入 Drive
 - Apps Script：唯一允許「帶密鑰寫 GitHub」的橋
 - GitHub：版本歷史與備援 raw
 - Firebase Hosting：正式 CDN（快取、自訂網域較方便）
 - App：只讀公開 URL，永不持有發布密鑰
-

3. 各環節如何「自動」起來？

3.1 內容端：覆寫同一個 Drive 檔

約定：

- 固定資料夾（例：【你的專案】資料）
- 固定檔名（例：knowledge_cards.json）
- 同一資料夾只能有一份同名檔（Drive 允許同名多檔；以 File ID 區分）

AI Skill 應明確要求：

- 先讀既有檔當底稿
- 合併後覆寫同一檔
- 禁止再建第二個同名檔
- 禁止在對話中要求 Token

3.2 Apps Script：定時讀 Drive → 寫 GitHub

腳本（見本倉庫 automation/apps-script/Code.gs）典型行為：

1. 用 DRIVE_FILE_ID 讀檔（不依檔名搜尋，避免抓到舊複本）
2. JSON.parse + schema 驗證
3. 與 GitHub 上現有檔比較：

– 若 version 相同且 id 集合相同 → 跳過（避免空 commit）

– 若張數變少或 version 倒退 → 拒絕（防清空）

1. 透過 GitHub Contents API 更新：

– knowledge_cards.json

– public/knowledge_cards.json（Hosting 靜態根目錄鏡像）

指令碼屬性（Script Properties）範例（隱碼）：

屬性	範例值（請替換）
GITHUB_TOKEN	github_pat_.....
DRIVE_FILE_ID	
GITHUB_OWNER	YOUR_GITHUB_OWNER
GITHUB_REPO	YOUR_PUBLIC_DATA_REPO
GITHUB_BRANCH	main

觸發器：時間驅動 → 呼叫 `syncCatalogFromDrive`（建議 15–60 分鐘）。

3.3 GitHub Actions：push 後部署 CDN

工作流（`.github/workflows/publish.yml`）在下列情況執行：

- push 到 main 且變更 JSON / Hosting 設定
- workflow_dispatch（手動）
- repository_dispatch（類型例：`catalog-update`，適合小檔備援路徑）

步驟概要：

1. Checkout
2. （若為 dispatch）把 payload 寫成兩份 JSON
3. 驗證 schema / 唯一 id
4. `firebase deploy --only hosting`（使用 Secret `FIREBASE_TOKEN`）

Actions Secret（隱碼）：

Secret	用途	範例
FIREBASE_TOKEN	<code>firebase login:ci</code> 產生的 CI token	1//.....

若 Secret 未設定，工作流可選擇 `soft-fail`，至少保留 GitHub raw 備援可用。

3.4 客戶端如何抓資料？

建議下載順序（本專案 App 實作概念）：

1. 主 CDN：`https://YOUR_PROJECT.web.app/knowledge_cards.json`
2. 備援 raw：`https://raw.githubusercontent.com/YOUR_OWNER/YOUR_REPO/main/knowledge_cards.json`
3. 內建 seed：App bundle 內一份底稿（離線也能用）

合併策略範例：以 `id` upsert，遠端新卡進來、舊卡不無故消失。

本倉庫公開實例（無密鑰）：

- CDN：https://wandanle-catalog.web.app/knowledge_cards.json
- Raw：https://raw.githubusercontent.com/jimmy77733/wandanle-catalog/main/knowledge_cards.json

4. 資料契約（為何驗證很重要）

自動化管線必須假設「上游偶爾會寫錯」。本專案根物件固定為：

```
{
  "version": 7,
  "updatedAt": "2026-08-01T07:26:49Z",
  "cards": [ /* ... */ ]
}
```

每張卡欄位：`id`，`title`，`content`，`category`，`sourceName`，`sourceURL`，`funFactValue`。

守門規則建議：

- 根鍵恰好三個，不多不少
- `category` 白名單
- `sourceURL` 允許 `null`，禁止 `""`
- `id` 全域唯一
- 新檔張數不得少於線上（防清空）
- `version` 不得倒退

把同樣規則寫在 **Apps Script** 與 **GitHub Action** 兩處，形成雙重防線。

5. 一次建置清單（可複用到其他專案）

A. 公開資料 repo（建議與 App 源碼分離）

- [] 新建 **Public** repo，只放資料與部署設定
- [] 根目錄與 `public/` 各放一份相同 JSON（方便 Hosting）
- [] 加入 `.github/workflows/publish.yml`
- [] 設定 Firebase 專案與 `FIREBASE_TOKEN` Secret

B. Drive 工作區

- [] 建立專用資料夾
- [] 放「唯一」資料檔，記下 `/d/FILE_ID/`
- [] 之後一律覆寫此檔，不另建同名檔

C. GitHub Fine-grained PAT

- [] 僅授權該資料 repo
- [] Contents : **Read and write**
- [] Token 只貼進 Apps Script 屬性，永不進 Skill / 公開 README

D. Apps Script

- [] 貼上轉發腳本
- [] 設定 Script Properties
- [] 手動跑通一次並授權
- [] 加上時間觸發器

E. 內容生產者 Skill / SOP

- [] 寫明：只寫 Drive、禁止 Token、禁止 push
- [] 寫明：覆寫既有檔、禁止同名複本
- [] 寫明：完整檔 = 舊資料 + 增量，version + 1

6. 驗收步驟

1. 在 Drive 把 version 提高並追加至少一筆合法資料
2. 手動執行 Apps Script (或等觸發)
3. 預期日誌：已更新 GitHub。version=N total=... (不是「跳過」)
4. 檢查 GitHub commits
5. 檢查 Actions → Publish 成功
6. 用 curl 比對 CDN 與 raw 的 version / 筆數

```
curl -sL "https://YOUR_PROJECT.web.app/YOUR_FILE.json" \  
  | python3 -c "import sys,json;d=json.load(sys.stdin);print(d['version'],len(d['cards']))"  
  
curl -sL "https://raw.githubusercontent.com/YOUR_OWNER/YOUR_REPO/main/YOUR_FILE.json" \  
  | python3 -c "import sys,json;d=json.load(sys.stdin);print(d['version'],len(d['cards']))"
```

7. 常見故障排查

現象	可能原因	處理
一直「相同 version/ids，跳過」	DRIVE_FILE_ID 仍指向舊檔；或 Drive 有兩個同名檔	刪複本；更新 FILE_ID 為最新那份
GitHub 有新 commit，CDN 仍舊	Action Firebase 步驟失敗 / Secret 空	查 Actions log；重設 FIREBASE_TOKEN
腳本拒絕覆寫	張數變少或 version 倒退	用完整底稿重建，勿只上傳增量
AI 一直「產生新檔」	Skill 仍要求 git push	改成 Drive-only Skill
dispatch 失敗	payload 過大（約 > 60KB）	改用 Contents API 路徑 (syncCatalogFromDrive)

8. 安全與隱私檢查清單（公開文件必過）

- [] 文件 / README / Skill 皆無完整 PAT、Firebase CI token、服務帳戶 JSON
- [] 截圖打碼：token、email、私人 Drive 檔名若含個資
- [] PAT 採 Fine-grained，repo 範圍最小化
- [] 公開 repo 不要放 App 源碼、後台金鑰、使用者資料
- [] App 內只存公開 CDN URL，不存發布密鑰

隱碼慣例建議：

```
GITHUB_TOKEN=github_pat_.....  
FIREBASE_TOKEN=1//.....  
DRIVE_FILE_ID=.....  
EMAIL=you.....@example.com
```

9. 套用到「其他類似系統」的對應表

本專案角色	你的專案可替換成
冷知識圖鑑 JSON	題庫、商品目錄、關卡表、多語字串包
Gemini Spark	任意只能寫雲端硬碟 / 不能持密鑰的 AI
Drive 約定檔	Notion export、Sheets→JSON、人工上傳匣
Apps Script 橋	Cloud Function + 排程、自架 worker（同樣持密鑰）
公開 catalog repo	任何 Public 資料鏡像 repo
Firebase Hosting	Cloudflare Pages、S3+CloudFront、GitHub Pages
iOS App 抓取	Android / Web / 後端快取層

核心不變：內容寫入私有中繼 → 受控橋樑持密鑰寫公開真相來源 → CI 部署 CDN → 客戶端只讀。

10. 本倉庫相關檔案索引

路徑	說明
knowledge_cards.json	公開圖鑑（根目錄）
public/knowledge_cards.json	Hosting 鏡像
.github/workflows/publish.yml	驗證 + Firebase 部署
automation/apps-script/Code.gs	Drive → GitHub 轉發腳本
automation/apps-script/README.md	設定步驟（維護者）
docs/catalog-automation-workflow.pdf	本文件 PDF 版

更短的維護者設定步驟：見 `../automation/apps-script/README.md`。

11. 結語

這套工作流的價值不在「某個產品名稱」，而在把三件事拆開：

- 1. 誰可以產出內容（可不持密鑰）
- 2. 誰可以發布（必須持最小權限密鑰）
- 3. 誰可以消費（只讀公開 URL）

當你下一個專案再次遇到「AI 產內容但不能碰 Token」時，直接複製此模式：Drive（或同等中繼）→ 排程橋 → Public repo → Hosting → Client。

文件產生於公開資料倉庫；不含私人 App 原始碼與真實密鑰。

本 PDF 由公開倉庫 docs/ 產生；不含私人 App 原始碼與真實密鑰。